本科毕业设计（论文）开题报告

一、课题基本信息

项目	内容
课题名称	面向代码智能体的开发任务上下文追踪与代码变更溯源系统设计与实现
英文名称	Design and Implementation of a Task-Context Tracking and Code-Change Provenance System for Code Agents
关键词	代码智能体；开发任务；上下文追踪；代码变更溯源；大语言模型
学院 / 专业	待填写
学生姓名 / 学号	待填写
指导教师 / 职称	待填写
课题来源	自拟（结合代码智能体方向的前沿问题与工程实践）
课题性质	工程设计（含研究性内容）
开题时间	2026 年 11 月

二、选题背景与意义

（一）研究背景

以大语言模型为决策核心的代码智能体（code agent）正在改变软件开发的组织方式。开发者不再逐行编写全部代码，而是用自然语言描述一个开发任务，由智能体自主完成代码检索、文件读写、命令与工具调用、测试执行与错误排查。SWE-bench 等基准印证了智能体能力的演进：该基准发布时表现最好的模型仅能解决 1.96% 的真实仓库 Issue[1]；自 Codex 将代码生成能力带入工程实践以来[2]，Claude Code、Cursor、Cline、OpenHands 等工具已进入日常开发流程，围绕该基准的评测与改进也持续进行[3]。

然而，智能体能力的提升并未同步解决“开发过程如何被理解和记忆”的问题。现有工具产生的记录天然是分散的：对话历史在一个面板，代码差异在另一个视图，工具调用与命令输出散落在终端或日志文件，测试结果属于另一套报告，版

本演化则另存于 Git。这种分散在一次开发任务被拉长、被打断或与其它任务并行时，会迅速转化为认知负担。

设想一个具体场景：开发者要求智能体“把订单模块的同步调用改为异步，并补上超时重试”，智能体在 40 分钟内修改了 8 个文件、执行了 6 次测试、中间还回退过一次方案。两天后开发者回到这个任务，需要回答五个问题：当前做到哪一步；这 8 个文件的修改分别对应哪句需求；哪几处修改还没有通过测试验证；中间为什么放弃了第一版方案；接下来还差什么。在现有工具下，回答这五个问题需要同时打开对话记录、逐个文件比对差异、翻查终端历史与测试报告，再把信息拼接起来；据笔者使用经验，这一过程通常需要十几分钟以上，且结论完整性取决于记忆是否可靠。

本课题正是针对这一场景：研究开发任务、对话内容、代码变更、工具执行与测试结果之间的关联与组织方法，并实现一个以“任务”为单位组织开发过程、支持代码变更溯源与任务上下文恢复的原型系统。

（二）研究意义

方法层面，本课题将提出一套面向智能体辅助开发过程的“任务—对话—变更—执行—测试”五元关联模型，以及在此基础上构建变更证据链的方法。现有工作多从单一数据类型出发（或研究对话，或研究变更，或研究执行轨迹），缺少把四类异构记录统一挂接到开发任务之下的表示方法，本课题可为开发过程的可解释性与可审计性研究提供参考。

工程层面，系统直接面向单个开发者的日常使用，目标是降低上下文维护成本、缩短任务恢复时间、提高代码审查与问题定位效率。这类“开发过程记忆”能力目前尚未见到成熟的轻量级实现，具有明确的实用价值。

教学与训练层面，课题覆盖需求分析、数据建模、事件采集、程序分析、前后端开发、实验设计与统计分析等完整环节，能够系统训练学生的工程实现能力与研究方法。

（三）核心概念界定

为避免歧义，本报告统一使用以下术语，其定义与选题说明文件保持一致。

开发任务（task）：开发者发起的一次有明确目标的开发工作单元，可包含多轮对话、多次文件修改、多次工具调用与多次测试执行。

任务上下文（task context）：与某个任务相关的对话、变更、执行、测试、决策与未完成事项的集合。

变更块（hunk）：一次代码修改中被连续修改或新增的最小代码区间，是本文溯源的基本粒度。

变更溯源（change provenance）：从一段代码变更出发，可追溯其触发来源、实施过程、验证依据与后续演化的证据链。

变更卡片（change card）：系统为每个变更块生成的记录单元，聚合上述四类证据。

五元关联模型：以开发任务为根，把对话（message）、变更（change）、工具执行（tool call）与测试（test run）四类记录挂接到任务之下，并由任务统一索引的表示方法；“五元”指任务、对话、变更、执行、测试五类实体。

三、国内外研究现状及发展趋势

（一）代码智能体与软件工程自动化

代码智能体的研究在 2021 年 Codex 之后快速推进[2]。SWE-bench[1] 以真实开源仓库的 Issue 为任务来源，给出了端到端的软件工程评测基准，其“从自然语言问题到可合入补丁”的设定成为后续工作的共同参照。SWE-agent[3] 提出智能体—计算机接口的设计理念，指出工具与反馈的呈现方式本身会影响智能体表现。OpenHands[4] 将智能体能力抽象为统一的动作空间，构建了可扩展的开放平台；CodeAct[5] 主张用可执行代码作为智能体的统一动作表示。在协作层面，MetaGPT[6] 以标准作业程序约束多智能体分工，ChatDev[7] 以角色化对话模拟软件开发流程。也有工作反其道而行，用不带复杂工具链的 Agentless 流程[8] 说明：在特定基准上，简洁的定位—修复—验证流水线同样有效。

国内研究同样活跃。已有综述系统梳理了大语言模型在软件工程各环节的应用形态与开放问题[9,10]，检索增强生成（RAG）在需求、编码、测试、维护中的应用也有专门整理[11]；此外，利用大语言模型自动生成故障复现测试用例的工作，

把“问题复现”这一环节自动化[12]。这些工作共同表明：大模型与智能体已深度介入开发过程的多个阶段。

但上述研究的目标函数是“任务能否被正确完成”，评价指标是补丁通过率或测试通过率；开发过程中的信息如何被组织、回放与理解，并不在其考察范围内。

（二）开发过程数据与细粒度代码变更

软件仓库挖掘（MSR）社区长期研究提交、缺陷、开发者行为等过程数据的采集与分析，Kagdi 等人的综述系统整理了面向软件演化的挖掘方法与分类体系[13]。在变更表示上，Robbes 与 Lanza 提出以“变更”而非“版本”为基本单位的演化分析方法，使细粒度的编辑操作成为一等公民[14]；GumTree[15] 基于抽象语法树做结构化差分，通过语法层面的匹配减少格式变动带来的噪声；CodeShovel[16] 在此基础上构造函数级的源代码变更历史，回答“这个方法是怎么演化成现在这样的”。

这类工作为本课题提供了两点启示：其一，变更应当以细粒度（变更块、函数、语句）而非文件为单位被记录和查询；其二，变更之间存在演化关系（重写、回退、迁移），需要显式建模。但其数据来源是版本库，不包含“人为什么这么改”的自然语言证据。

（三）过程溯源与“为什么”类解释

溯源（provenance）在数据库领域已有完整理论，Cheney 等人从“为什么、如何、何处”三个维度刻画了溯源的含义与计算方式[17]。在程序理解方向，Ko 与 Myers 的 Whyline[18] 通过记录执行轨迹，让开发者可以就程序行为直接追问“为什么”与“为什么不”，是把证据链转化为可交互界面的经典范例。noWorkflow[19] 面向脚本型科学计算，捕获执行过程中的变量、文件与依赖，支持事后溯源查询。

这些工作证明了以证据链组织过程信息的有效性，但其溯源对象是程序执行本身，而非“人类用自然语言下达指令、智能体实施修改”这一新型过程，缺少对“指令—修改—验证”链路的建模。

（四）任务中断与开发者认知负担

Parnin 与 Rugaber 通过实证研究刻画了开发者任务中断后的恢复策略，指出恢复质量高度依赖可用线索的丰富程度，且开发者普遍采用回顾最近编辑、重读任务描述、重新运行等策略重建上下文[20]。在人机协作方向，Barke 等人对使用代码生成工具的程序员进行观察研究，区分了加速型与探索型两种交互模式[21]；Vaithilingam 等人发现开发者对代码生成工具的期望与实际体验存在落差，其中“难以判断生成代码是否正确”是主要痛点之一[22]。

这些研究从人的角度印证了本课题的动机：恢复上下文与判断修改可信度是真实存在的负担，而现有工具的线索组织方式并不适配这一需求。

（五）现有工具的能力与局限

工程侧的工具可分为三类。第一类是智能体客户端本身，如 Claude Code[23]、Cursor[24]、Cline[25]，它们提供会话历史与差异视图，但组织单位是“会话”或“文件”，跨会话的任务连续性弱。第二类是 LLM 应用可观测性平台，如 Langfuse[26]，能够记录调用链、提示词与耗时，OpenTelemetry 也已提出生成式 AI 的语义约定[27]，但其面向调用调试与成本监控，不与代码变更、测试结果做语义级对齐。第三类是版本控制与 IDE 本地历史，它们记录文件快照，却不记录修改动因与验证依据。

三类工具在各自的目标上都有完整实现，但彼此之间缺少一层以任务为中心的关联与索引。

（六）研究述评

本节五个小节可以归为三个方向：第（一）节对应“智能体能力”，回答能不能做；第（二）节对应“变更分析”，回答改了什么；第（三）（四）节分别从机器与人的角度回答“为什么需要证据”；第（五）节说明工程工具现状。三个方向各有积累，但存在一个明确的空白：**缺少以开发任务为中心、跨对话/代码/执行/测试四类异构记录的统一关联模型，以及面向开发者的可交互溯源与上下文恢复方法**。本课题即定位于此。

四、研究目标与研究内容

（一）研究目标

构建一个面向单个开发者的原型系统，使开发者能够围绕任务、文件或代码片段，查看相关的对话、修改、执行与测试记录，并在任务中断后快速恢复上下文。具体目标为：

1. 提出并实现五元关联模型与相应的事件表示；

2. 实现从代码变更反向追溯触发来源、实施过程与验证依据的证据链查询；

3. 实现任务边界的自动判定与任务上下文的快照、恢复；

4. 实现基于大语言模型的任务摘要与关联补全，并保证结论可回溯；

5. 在典型编程任务上完成评估，量化系统在代码理解、修改原因分析、问题定位与任务衔接四个维度的作用。

各项指标的口径、数据来源与达标线见表 7-1。考虑到小样本条件下难以稳定检出固定比例的用时下降，本研究对用时类指标同时报告效应量与 95% 置信区间，而不以单一百分比作为达标依据。

（二）研究内容

（1）开发任务过程的信息模型与统一事件表示。 抽象开发任务、对话轮次、消息、工具调用、文件变更、测试运行、测试用例结果与任务检查点等实体及其关系，设计可扩展的事件模式与存储结构，使不同来源的记录能够被统一表示。

（2）上下文采集、任务边界判定与信息关联。 实现会话、工具调用、文件系统变更与测试结果的采集；研究任务边界的判定规则（依据空闲时长、目标切换、显式指令与提交动作综合判定）；研究基于时间窗口、显式引用（路径、符号、行号、命令、测试名）与代码位置映射的关联方法，并处理行号漂移、重写与回退；对无法自动关联的记录给出置信度并支持人工确认。

（3）代码变更溯源与证据链构建。 以变更块为粒度生成变更卡片，记录触发来源、实施过程、修改前后快照、验证依据与后续演化，支持“代码→原因→证据”与“对话/测试→受影响代码”的双向查询。

（4）任务上下文追踪界面与任务续接功能。 实现任务中心、任务时间线、文件与代码片段视角、变更证据侧栏，以及任务中断后的上下文恢复视图。

（5）大语言模型辅助的摘要与关联。 利用大语言模型生成任务摘要、归纳变更意图、整理开发进展并补全缺失关联，要求输出绑定原始证据标识以抑制幻觉，并对其准确性做专项评估。

五、拟解决的关键问题

关键问题一：异构记录的统一表示与跨源实体链接。 对话是文本流，变更是代码位置区间，工具调用是结构化事件，测试结果是层级化结果集，四者的时间粒度与标识体系均不相同。如何设计一套既能容纳四类记录、又不过度耦合于某一具体智能体的表示方法，是系统成立的前提。

关键问题二：变更与对话的可靠关联。 智能体通常不会在修改代码时显式引用“这是响应第几条消息”，关联必须依靠启发式推断。难点在于：一次修改可能由多轮对话共同促成；行号会随修改漂移；同一文件可能被并行任务交替修改。如何在保证召回的同时控制误关联，并让开发者能识别关联的可信程度，是溯源质量的核心。

关键问题三：证据链的可信呈现与 LLM 幻觉抑制。 引入大语言模型生成摘要与关联建议可降低使用门槛，但模型可能编造不存在的因果关系。需要设计证据绑定与可回溯校验机制：模型的每一条结论都必须指向真实存在且内容相关的消息、变更或测试记录，否则不予展示。

关键问题四：任务边界的划分与恢复上下文的“最小充分性”。 什么算一个任务的开始与结束，在连续对话中并不明确；恢复上下文时，呈现全部信息等于没有恢复，呈现过少又不足以支撑决策。前者由研究内容（2）的任务边界判定规则承接，后者由研究内容（4）的恢复视图与第七节的恢复用时、理解正确率指标共同检验。

六、研究方法与技术路线

（一）研究方法

文献研究法。 系统梳理代码智能体、软件仓库挖掘、过程溯源与人机交互四个方向的文献，明确研究空白与可复用方法。

原型实现法。 以可运行原型为研究载体，通过迭代实现验证设计假设，避免停留在纸面方案。

实验研究法。 在固定开源仓库上构造典型编程任务，采集真实开发过程数据，通过对照实验量化系统效果。

问卷调查与访谈法。 通过 NASA-TLX 认知负荷量表[28]、SUS 可用性量表[29] 与半结构化访谈，获取开发者主观反馈，补充客观指标的不足。

（二）技术路线

研究按“需求与调研—模型设计—采集实现—关联与溯源—界面与续接—评估验证”六个阶段推进，前一阶段的产出构成后一阶段的输入，并在每个阶段设置可检验的里程碑。技术路线如图 6-1 所示。

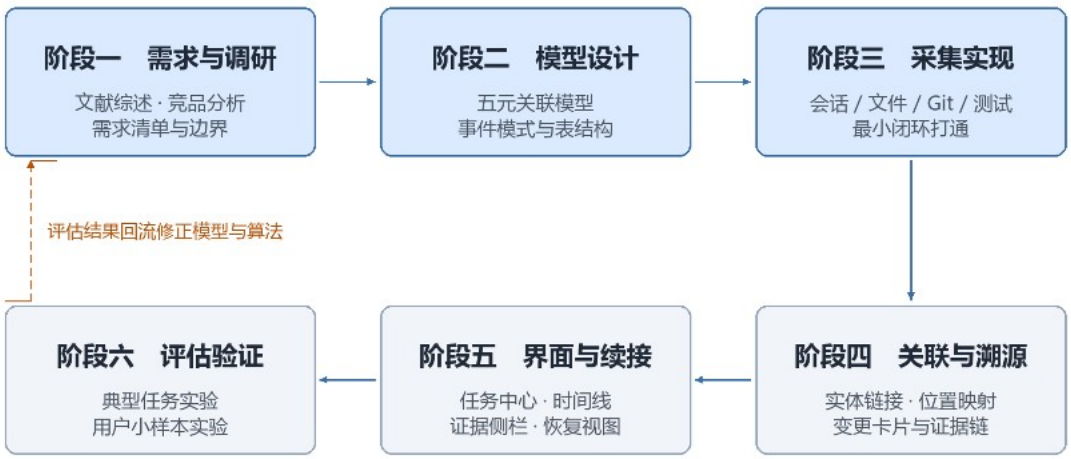


图 6-1 研究技术路线

第一阶段通过文献调研与竞品分析确定需求边界；第二阶段产出实体关系模型与事件模式，是后续工作的契约；第三阶段实现采集器，先打通“一次会话→文件变更→测试结果”的最小闭环；第四阶段在闭环数据上实现关联算法与证据链查

询；第五阶段把查询能力转化为界面与恢复视图；第六阶段用典型任务与用户实验检验效果，并回流修正模型与算法。

（三）系统总体架构

系统采用四层架构，如图 6-2 所示。接入层从智能体会话日志、文件系统监听、Git 与测试输出中抽取原始事件；模型与存储层把原始事件规范化为统一事件并写入实体关系表；分析与关联层完成实体链接、证据链构建与 LLM 辅助分析；交互层提供任务中心、时间线、代码视角与证据侧栏。LLM 服务作为旁路能力被分析与交互层调用，所有输出经证据校验后才进入界面。

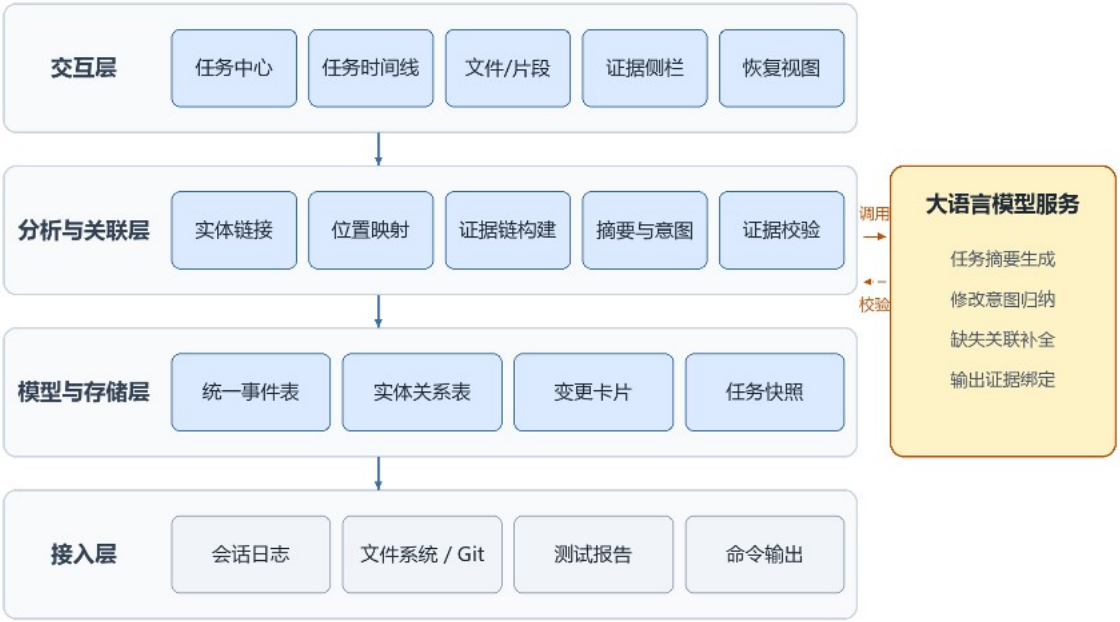


图 6-2 系统总体架构

（四）关键方法与研究内容的对应

关键方法的设计要点及其与研究内容的对应关系如下，具体实现细节在论文中展开。

信息模型以开发任务为根实体，向下挂接对话与消息，横向关联由消息触发的工具调用、由工具调用产生的文件变更、以及由命令触发的测试运行；事件采用 JSON 表示，字段设计参考 OpenTelemetry 生成式 AI 语义约定[27]，并保留原始字段以容纳不同智能体的格式差异，对应内容（1）。

关联算法采用“规则打分—阈值判定—人工确认”三段式：以时间邻近度、显式引用匹配、代码位置映射与文本相似度四类规则加权得到关联置信度，高于阈值自动关联，中间区间标注为待确认，对应内容（2）。任务边界判定与关联共用同一套事件特征，先按空闲时长与目标切换切分候选边界，再用提交动作与显式指令校正，对应内容（2）。

变更卡片聚合触发消息、相关工具调用、修改前后代码、验证该变更的测试结果与后续演化关系；演化关系通过变更的父子链接维护，使“这段代码怎么一步步变成现在这样”可被完整还原，对应内容（3）。

任务快照记录已完成事项（有通过测试支撑的变更）、未完成事项（已提出未实施的计划、失败或未运行的测试）与当前工作位置，恢复视图以摘要加清单呈现并支持跳转到证据，对应内容（4）。

LLM 承担任务摘要、变更意图归纳与缺失关联候选生成三项任务，通过结构化输出、证据 id 后置校验与证据不足时弃答三重约束抑制幻觉，对应内容（5）。

七、实验与评估方案

（一）实验任务集与数据构造

在 2—3 个中小规模开源仓库（TypeScript 与 Python 各一至两个）上构造 15—20 个典型编程任务，覆盖缺陷修复、功能新增、代码重构、测试补充、依赖升级五类。任务由本人在内的两名开发者使用同一款代码智能体完成，全过程记录会话、变更、执行与测试数据，形成可复现的实验数据集。

变更与对话的关联关系、以及任务边界，由两名标注者独立标注，不一致处协商确定；标注一致性以 Cohen’s κ 报告，κ 低于 0.6 时修订标注规范后重新标注。计划标注 30 个任务边界与约 400 条变更—消息候选对。

（二）评价指标

指标的口径、数据来源与达标线见表 7-1。达标线为预期目标，最终以实测值与置信区间报告，不以达标线替代结论。

表 7-1 评价指标定义与预期达标线

指标	定义与口径	数据来源	预期达标线

关联准确率 / 召回率 / F1	以变更块为单元，系统给出的变更—消息关联与金标准的重合度	双人标注数据	F1 ≥ 0.75
溯源查询正确率	给定代码位置，返回的证据链是否包含金标准中的触发消息与验证测试	20 条预设查询	≥ 0.80
任务边界判定 F1	系统切分的任务边界与人工标注边界的重合度	30 个标注边界	≥ 0.80
任务恢复用时	受试者回答五个恢复性问题所需时间（秒）	实验记录	与基线比较报告效应量
问题定位用时	受试者定位“哪次修改引入了问题”所需时间	实验记录	与基线比较报告效应量
理解正确率	受试者对“这段代码为什么改”的回答与金标准的一致程度（0—5 分）	双人评分	均值提升并报告区间
认知负荷	NASA-TLX 加权得分	量表[28]	低于基线
可用性	SUS 得分	量表[29]	≥ 70
摘要与关联幻觉率	100 条 LLM 结论中引用不存在或不相关证据的比例	人工判定	≤ 5%
响应时间	万级事件下时间线与溯源查询耗时	性能测试脚本	≤ 1 s

（三）对比基线与实验设计

基线为现有工具组合：Git 日志与差异、智能体原始会话记录、测试报告，三者分别查看。

实验分两部分。**实验一（客观评测）** 使用自采数据，评测关联、溯源与任务边界判定，无需受试者参与；同时用 100 条 LLM 结论评估幻觉率。**实验二（受试者实验）** 面向恢复与理解：受试者作为“第三方接手者”，阅读他人（本人或协助者）产生的任务上下文，回答恢复性问题。这一设定与“自己中断后回来”并不

完全等价，属于对恢复能力的保守估计；作为补充，另招募 3—5 名受试者用自己产生的任务做自中断对照，比较两种情形下的差异。

实验二采用被试内设计，同一受试者在基线与系统两种条件下完成等价任务，任务顺序随机化以抵消学习效应。受试者规模计划 8—12 名具备编程经验的同专业学生；按配对设计、$\alpha = 0.05$、检验力 0.8、预期效应量 $d = 0.8$ 估算约需 15 人，受招募条件限制适当缩小样本，因此结论以效应量与置信区间表述，定位为初步证据。

（四）数据分析方法

连续型指标在正态性检验通过时采用配对 t 检验，否则采用 Wilcoxon 符号秩检验；同时报告效应量（Cohen's d 或 r）与 95% 置信区间。分类指标报告准确率、召回率与 F1，并给出 κ 值。显著性水平取 0.05，多重比较时作相应校正。

八、可行性分析

（一）技术可行性

所需技术均为成熟方案：前端采用 React 与 Vite，后端与采集端采用 TypeScript 或 Python，存储采用 SQLite（单机单用户场景足够，且便于答辩现场离线演示），代码解析采用 unified diff 与 tree-sitter[30]，LLM 通过 OpenAI 兼容接口调用。系统不涉及模型训练与分布式部署，技术风险可控。

（二）数据与实验条件

主流代码智能体以结构化日志（如 JSONL）记录会话与工具调用，文件变更可由文件系统监听与 Git 共同捕获，测试结果可由测试框架的 JSON 报告获得，三类数据均可采集。实验所需仓库为公开开源项目，任务由本人构造，不依赖涉密或受限数据。

（三）时间与工作量

课题周期为 2026 年 11 月至 2027 年 5 月，共 7 个月。按技术路线划分的六个阶段中，采集与关联阶段工作量最大，已在进度安排中预留缓冲；界面阶段采用最小可用优先策略，确保核心功能先跑通。

（四）风险与应对

表 8-1 主要风险与应对措施

风险	影响	应对措施
不同智能体事件格式差异大，采集适配成本高	进度延误	先支持一种来源（CLI 日志 + Git），采用适配器模式，其余来源按需扩展
关联准确率不达预期	溯源不可信	引入人工确认入口；LLM 仅提供候选；如实报告置信度分布与 κ 值
界面开发工作量超预期	影响论文进度	采用最小可用界面，优先保证任务时间线与证据侧栏两个核心视图
受试者招募不足	评估不充分	以客观指标为主；样本不足时报告置信区间并明确说明局限
行号漂移处理复杂	位置映射失效	以变更 id 为主键、行号为辅助；无法映射时降级为文件级关联并标注

九、创新点与特色

1. **以任务为中心的组织方式**。相比以“会话”为单位的智能体客户端和以“文件”为单位的 IDE 历史，本系统把对话、变更、工具调用与测试统一挂接到开发任务之下，形成可查询的任务上下文图。可验证差异：同一任务跨多次会话时，系统仍能返回完整的上下文集合。

2. **细粒度的变更证据链**。相比 Git 只记录“改了什么”、Langfuse 类平台只记录“调用了什么”，本系统以变更块为单位同时记录触发来源、验证依据与后续演化，可直接回答“为什么改、依据是什么、还有效吗”。可验证差异：给定任一变更块，能返回至少一条触发消息与一次相关测试运行。

3. **面向中断恢复的上下文重建**。相比聊天历史的顺序回放，本系统以任务快照与“最小充分”的恢复视图呈现已完成、未完成与待验证事项。可验证差异：恢复用时与理解正确率相对基线的变化可被测量。

4. **可回溯约束下的 LLM 辅助**。相比直接让模型总结对话，本系统要求每条结论绑定真实证据 id，证据不足时弃答。可验证差异：幻觉率可被人工判定并量化。

十、进度安排

各阶段的工作内容与阶段成果见表 10-1，时间关系如图 10-1 所示。

表 10-1 研究进度安排

阶段	时间	主要工作	阶段成果
一	2026.11.02—11.15	选题确认，文献调研与竞品分析	文献综述、需求清单
二	2026.11.16—11.30	撰写开题报告，确定技术路线	开题报告与答辩
三	2026.12.01—12.20	信息模型与事件模式设计，采集可行性验证	设计文档、最小闭环
四	2026.12.21—2027.01.10	采集模块、任务边界判定与关联算法	采集与关联原型
五	2027.01.11—01.20	中期检查材料整理	中期检查报告
六	2027.01.21—02.28	溯源查询、证据链与界面开发	原型 v0.5
七	2027.03.01—03.20	任务续接、LLM 辅助与关联	原型 v0.9
八	2027.03.21—04.10	典型任务实验与用户小样本实验	实验报告与数据
九	2027.04.11—04.30	撰写论文初稿	论文初稿
十	2027.05.01—05.15	修改、查重与送审	论文定稿
十一	2027.05.16—05.31	答辩准备与答辩	答辩材料

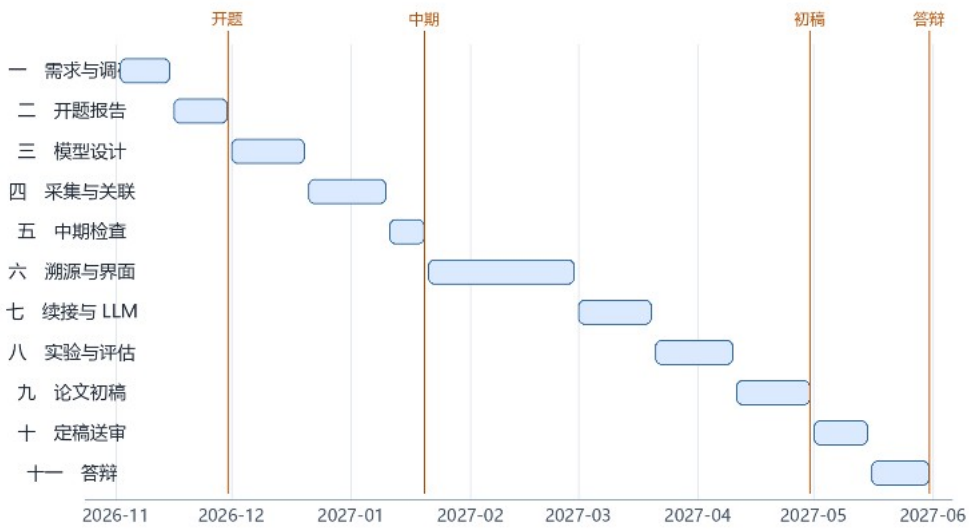


图 10-1 研究进度甘特图

十一、预期成果

1. 可运行的原型系统一套，含源代码、部署与使用说明；
2. 毕业设计（论文）一份，约 1.5—2 万字；
3. 文献综述、外文资料翻译（按学院要求）；
4. 系统设计文档与测试、评估报告，含实验数据与复现脚本；
5. 典型编程任务的演示视频与答辩幻灯片；
6. （可选）经标注的开发过程事件样例数据集。

十二、主要参考文献

[1] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: can language models resolve real-world GitHub issues?[C]//Proceedings of the 12th International Conference on Learning Representations (ICLR 2024). 2024. arXiv: 2310.06770.

[2] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code[EB/OL]. (2021-07-07)[2026-10-27]. https://arxiv.org/abs/2107.03374.

[3] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37 (NeurIPS 2024). 2024. arXiv: 2405.15793.

[4] WANG X, LI B, SONG Y, et al. OpenHands: an open platform for AI software developers as generalist agents[C]//Proceedings of the 13th International Conference on Learning Representations (ICLR 2025). 2025. arXiv: 2407.16741.

[5] WANG X, CHEN Y, YUAN L, et al. Executable code actions elicit better LLM agents[C]//Proceedings of the 41st International Conference on Machine Learning (ICML 2024). 2024. arXiv: 2402.01030.

[6] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: meta programming for a multi-agent collaborative framework[C]//Proceedings of the 12th International Conference on Learning Representations (ICLR 2024). 2024. arXiv: 2308.00352.

[7] QIAN C, LIU W, LIU H, et al. ChatDev: communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. 2024: 15174-15186. DOI: 10.18653/v1/2024.acl-long.810.

[8] XIA C S, DENG Y, DUNN S, et al. Agentless: demystifying LLM-based software engineering agents[EB/OL]. (2024-07-01)[2026-10-27]. https://arxiv.org/abs/2407.01489.

[9] HOU X, ZHAO Y, LIU Y, et al. Large language models for software engineering: a systematic literature review[EB/OL]. (2023-08-20)[2026-10-27]. https://arxiv.org/abs/2308.10620.

[10] FAN A, GOKKAYA B, HARMAN M, et al. Large language models for software engineering: survey and open problems[EB/OL]. (2023-10-05)[2026-10-27]. https://arxiv.org/abs/2310.03533.

[11] 张犬俊, 谢杨, 房春荣, 等. 检索增强生成在软件工程中的应用综述[J]. 软件学报, 2026, 37(3): 1316-1339. DOI: 10.13328/j.cnki.jos.007567.

[12] 汪莹, 字千成, 彭鑫, 等. 基于大语言模型的故障复现测试用例生成方法[J]. 软件学报, 2026, 37(4): 1690-1714. DOI: 10.13328/j.cnki.jos.007474.

[13] KAGDI H, COLLARD M L, MALETIC J I. A survey and taxonomy of approaches for mining software repositories in the context of software evolution[J]. Journal of Software Maintenance and Evolution: Research and Practice, 2007, 19(2): 77-131. DOI: 10.1002/smr.344.

[14] ROBBES R, LANZA M. A change-based approach to software evolution[J]. Electronic Notes in Theoretical Computer Science, 2007, 166: 93-109. DOI: 10.1016/j.entcs.2006.06.015.

[15] FALLERI J R, MORANDAT F, BLANC X, et al. Fine-grained and accurate source code differencing[C]//Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE 2014). 2014: 313-324. DOI: 10.1145/2642937.2642982.

[16] GRUND F, CHOWDHURY S A, BRADLEY N C, et al. CodeShovel: constructing method-level source code histories[C]//Proceedings of the 43rd International Conference on Software Engineering (ICSE 2021). 2021: 1510-1522. DOI: 10.1109/ICSE43902.2021.00135.

[17] CHENEY J, CHITICARIU L, TAN W C. Provenance in databases: why, how, and where[J]. Foundations and Trends in Databases, 2009, 1(4): 379-474. DOI: 10.1561/1900000006.

[18] KO A J, MYERS B A. Designing the Whyline: a debugging interface for asking questions about program behavior[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2004). 2004: 151-158. DOI: 10.1145/985692.985712.

[19] PIMENTEL J F, MURTA L, BRAGANHOLO V, et al. noWorkflow: a tool for collecting, analyzing, and managing provenance from Python scripts[J]. Proceedings of the VLDB Endowment, 2017, 10(12): 1841-1844. DOI: 10.14778/3137765.3137789.

[20] PARNIN C, RUGABER S. Resumption strategies for interrupted programming tasks[J]. Software Quality Journal, 2011, 19(1): 5-34. DOI: 10.1007/s11219-010-9104-9.

[21] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: how programmers interact with code-generating models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI: 10.1145/3586030.

[22] VAITHILINGAM P, ZHANG T, SO S. Expectation vs. experience: evaluating the usability of code generation tools powered by large language models[C]//CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI EA 2022). 2022: 1-7. DOI: 10.1145/3491101.3519665.

[23] ANTHROPIC. Claude Code documentation: overview[EB/OL]. [2026-10-27]. https://code.claude.com/docs/en/overview.

[24] CURSOR. Cursor documentation[EB/OL]. [2026-10-27]. https://cursor.com/docs.

[25] CLINE. Cline: AI coding, open source and open choice[EB/OL]. [2026-10-27]. https://cline.bot.

[26] LANGFUSE. Observability overview[EB/OL]. [2026-10-27]. https://langfuse.com/docs/observability/overview.

[27] OPENTELEMETRY. Generative AI semantic conventions[EB/OL]. [2026-10-27]. https://opentelemetry.io/docs/specs/semconv/gen-ai/.

[28] HART S G, STAVELAND L E. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research[J]. Advances in Psychology, 1988, 52: 139-183. DOI: 10.1016/S0166-4115(08)62386-9.

[29] BROOKE J. SUS: a quick and dirty usability scale[M]//JORDAN P W, THOMAS B, WEERDMEESTER B A, et al. Usability Evaluation in Industry. London: Taylor and Francis, 1996: 189-194.

[30] TREE-SITTER. Tree-sitter documentation[EB/OL]. [2026-10-27]. https://tree-sitter.github.io/tree-sitter/.

十三、开题审查意见

栏目	内容	签字 / 日期
指导教师意见		
开题审查小组意见		
学院意见		

学生（签名）：_____　　指导教师（签名）：_____

日期：_____年____月____日